

MedAI — Complete Project Flow

From Raw Medical Text → Tokenization → Qwen2.5-7B → QLoRA → Fine-Tuning → Evaluation → RAG → FAISS → Generation → Gradio Deployment

Purpose: A detailed, interview-oriented explanation of the complete medical chatbot pipeline. Each stage explains what the concept means, what happens internally, how it is used with the MedQuAD dataset, an example, and why the next stage is required.

Project: MedAI — Healthcare Assistant Medical Chatbot

Core model: Qwen2.5-7B-Instruct

Fine-tuning: QLoRA (4-bit quantized frozen base + trainable LoRA adapters)

Dataset: MedQuAD-style medical Question/Answer pairs

RAG: Embeddings + FAISS vector retrieval + retrieved context

Interface: Gradio

Important mental model

RAW MEDICAL TEXT

↓

Question/Answer formatting

↓

Qwen tokenizer

↓

Token IDs + attention mask

↓

Qwen embedding layer

↓

Transformer blocks

↓

Self-attention (Q/K/V) + FFN

↓

Frozen 4-bit base weights + trainable LoRA

↓

Final hidden states

↓

LM Head

↓

Logits

↓

Vocabulary probabilities

↓

Next-token selection

↓

Generated text

TRAINING:

Prediction + target → Cross-Entropy Loss → Backpropagation → 8-bit AdamW → update LoRA A/B only

RAG:

Documents → chunks → embeddings → FAISS

User query → query embedding → FAISS → top-K chunks

→ query + context → Qwen → final answer

1. What Are We Building?

- **LLM:** Large Language Model — a neural network trained to predict the next token in text.
- **Qwen2.5-7B-Instruct:** Alibaba's instruction-following causal language model with roughly 7 billion parameters.
- **MedQuAD:** a medical question-answer dataset used to teach the model medical-domain answering behavior.
- **QLoRA:** Quantized Low-Rank Adaptation — 4-bit base-model quantization plus small trainable LoRA adapters.
- **RAG:** Retrieval-Augmented Generation — retrieve relevant external text and give it to the LLM as context before generation.
- **FAISS:** Facebook AI Similarity Search — a library for fast similarity search over vectors.
- **Gradio:** a Python library for creating a simple web UI around an ML model.

Why this architecture? Fine-tuning adapts the model's behavior to medical Q&A, while RAG supplies relevant source information at query time. Together, they separate *behavior adaptation* from *retrievable knowledge*.

2. Dataset: MedQuAD and Cleaning

Definition: A dataset is a collection of examples used to train, validate, or test a machine-learning system.

Our working table contains **qtype**, **Question**, and **Answer**. After removing duplicate Question+Answer pairs, the working dataset had about **16,359 examples**.

Cleaning: remove missing questions/answers, strip whitespace, remove empty strings, and remove duplicate pairs.

```
df = df.dropna(subset=["Question", "Answer"])
df["Question"] = df["Question"].str.strip()
df["Answer"] = df["Answer"].str.strip()
df = df[(df["Question"] != "") & (df["Answer"] != "")]
df = df.drop_duplicates(
    subset=["Question", "Answer"]
).reset_index(drop=True)
```

Example: Question = "Who is at risk for Lymphocytic Choriomeningitis (LCM)?" Answer = a medical explanation about exposure to infected rodents.

Why next? Neural networks cannot directly train on an unstructured spreadsheet. We first need to convert each Q/A example into the conversational text format expected by the instruction-tuned model.

3. Instruction / Chat Formatting

Definition: Chat formatting converts a question-answer pair into a structured user-assistant conversation.

For Qwen-Instruct, the preferred approach is to use Qwen's built-in **chat template**, which inserts the model's expected role/special tokens.

```
messages = [
    {"role": "user", "content": row["Question"]},
    {"role": "assistant", "content": row["Answer"]}
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=False
)
```

Internal idea: the model must learn that one part is the user's request and the following part is the assistant's response. The template makes this structure explicit.

Dataset use: every MedQuAD Q/A pair becomes one training conversation.

Why next? The formatted text must be converted into machine-readable token IDs before Qwen can process it.

4. Tokenization — Text → Tokens → Token IDs

Definition: Tokenization splits text into the vocabulary units used by an LLM. Tokens are not necessarily whole words; they may be words, subwords, punctuation, or other pieces.

Qwen tokenizer: Qwen2.5 comes with its own tokenizer and vocabulary. We normally do not replace it with Word2Vec/GloVe embeddings.

Typical subword idea: a rare word can be represented by several pieces rather than requiring one vocabulary entry for every possible word.

```
inputs = tokenizer(
    text,
    truncation=True,
    max_length=2048,
    padding="max_length"
)
```

Outputs: **input_ids** (integer IDs for vocabulary tokens) and **attention_mask** (1 for real tokens, 0 for padding).

Important: token IDs are identifiers, not semantic vectors. ID 7421 does not itself mean “anemia”; the embedding layer gives that ID a learned vector.

Truncation: if a sequence exceeds 2048 tokens, it is cut to the maximum length. **Padding:** shorter sequences are padded to the selected length.

Why next? Transformer layers operate on vectors, not integer token IDs. Therefore token IDs must enter Qwen's learned embedding layer.

5. Qwen Embedding Layer — Token IDs → Dense Vectors

Definition: An embedding is a dense numerical vector learned to represent a token in a continuous vector space.

Conceptually, Qwen has an embedding matrix **E** with shape approximately `vocabulary_size × hidden_size`. For each token ID, the model selects the corresponding row of **E**.

```
token_ids
↓
Embedding matrix E
↓
one vector per token
↓
sequence of hidden vectors
```

Important distinction: Word2Vec/GloVe are external embedding-learning techniques. In Qwen, the input embedding matrix is part of the pretrained model and is learned during the model's original pretraining.

Example: “anemia” is tokenized into one or more token IDs; each ID indexes Qwen's embedding table, producing a vector of the model's hidden dimension.

Why next? The vectors now have the correct shape and can be processed by the Transformer blocks.

6. Positional Information and RoPE

Problem: self-attention by itself does not inherently know the order of tokens.

RoPE: Rotary Positional Embedding — a positional method used in Qwen-family Transformers that rotates query/key representations according to token position.

Conceptually: token embeddings represent *what* the token is; positional information helps the model represent *where* it occurs in the sequence.

Why next? The model now has token representations with positional information and can perform contextual self-attention.

7. Transformer Blocks — Where Fine-Tuning Acts

Definition: A Transformer block is a neural-network module that repeatedly transforms token representations using self-attention and a feed-forward network (FFN), with normalization and residual connections.

Qwen2.5-7B has a stack of Transformer blocks. Earlier discussion used 28 blocks for the model configuration.

```
For each block:
Hidden states
↓
Normalization
↓
Self-Attention
↓
Residual connection
↓
Normalization
↓
FFN
↓
Residual connection
↓
Next block
```

Self-attention: allows each token to use information from other relevant tokens. For a causal language model, the attention pattern prevents a token from looking at future target tokens during generation/training.

FFN: Feed-Forward Network — a nonlinear transformation applied independently to each token position after attention.

Residual connection: adds the block input back to the transformed representation, helping information and gradients flow through deep networks.

Why next? We need to understand Q/K/V because these are the core computations inside self-attention and are also common places for LoRA adapters.

8. Self-Attention — Q, K, V

Q = Query: represents what the current token is looking for. **K = Key:** represents what each token offers for matching. **V = Value:** contains the information that will be combined.

```
Q = X WQ
K = X WK
V = X WV

Attention(Q, K, V)
= softmax(QKT / √dk) V
```

Attention softmax: this is the first important softmax in the pipeline. It converts attention scores into weights that sum to one across the allowed positions.

Example: in “iron deficiency causes anemia”, the representation of “anemia” can attend strongly to “iron” and “deficiency” because those words are informative in context.

Two different softmaxes: attention softmax decides *which tokens to attend to*; the later vocabulary softmax decides *which vocabulary token to predict next*.

9. QLoRA — 4-bit Quantization + LoRA

Definition: QLoRA (Quantized Low-Rank Adaptation) combines a memory-efficient quantized frozen base model with trainable low-rank adapters.

Part A — 4-bit quantization: pretrained floating-point base weights are represented in a compact 4-bit format. Four bits provide $2^4 = 16$ possible codes per quantized value. In QLoRA, NF4 (NormalFloat4) is commonly used for weight quantization.

Important: 4-bit quantization applies to model weights, not to the output probability distribution. Intermediate activations such as Q/K/V are normally computed using a higher-precision compute dtype such as FP16/BF16 depending on configuration.

Part B — LoRA: Low-Rank Adaptation does not directly rewrite the full pretrained matrix. It adds a small trainable update:

```
W_effective = W_base + ΔW
ΔW = B A
```

If W is 3584×3584 , an illustrative LoRA rank $r=16$ uses A of size 16×3584 and B of size 3584×16 . This is dramatically smaller than training all 3584×3584 parameters.

Where: LoRA is attached to selected linear layers, often attention projections such as q_proj and v_proj ; exact target modules depend on the model architecture/configuration.

Frozen vs trainable: W_base is frozen; LoRA A and B are trainable.

Why next? The model must now be trained on medical examples so the LoRA adapters learn the desired domain behavior.

10. Training Forward Pass

For a MedQuAD training example, the tokenized sequence is passed through the entire Qwen model.

```
Input IDs
↓
Qwen embeddings
↓
Transformer block 1
↓
...
↓
Transformer block 28
↓
Final hidden states
↓
LM head
↓
Logits
```

LM head: Language Model head — a final projection from the hidden dimension to the vocabulary dimension. It produces one raw score (logit) for every vocabulary token at each position.

Why next? Raw logits are scores, not probabilities. We need probabilities and a loss to measure how well the model predicts the training target.

11. Logits → Vocabulary Probabilities → Cross-Entropy Loss

Logits: unnormalized scores produced by the LM head.

Vocabulary softmax: converts logits into a probability distribution over the vocabulary.

```
logits
↓
softmax
↓
P(token_1), P(token_2), ... P(token_V)
```

Suppose the correct next token is “iron”. If the model assigns $P(\text{iron})=0.90$, the loss for that token is small; if $P(\text{iron})=0.01$, the loss is large.

```
Loss = -log(P(correct token))
```

Teacher forcing: during causal language-model training, the known target sequence is used to calculate the loss for the next-token predictions. The model is trained to predict each target token from the preceding context.

Why next? The loss tells us how wrong the model was. Backpropagation uses that loss to calculate gradients.

12. Backpropagation — Gradients for LoRA

Definition: Backpropagation computes how the loss changes with respect to trainable parameters.

```
Loss
↓
Backpropagation
↓
∂Loss/∂A and ∂Loss/∂B
↓
LoRA gradients
```

Because the pretrained Qwen parameters are frozen, they are not updated by the optimizer. The trainable LoRA parameters receive gradients and are optimized.

Gradient intuition: a gradient tells the optimizer which direction a parameter should move to reduce the loss.

Why next? Gradients tell us the required direction, but an optimizer is needed to turn gradients into actual parameter updates.

13. 8-bit AdamW — Updating LoRA Weights

AdamW: an adaptive optimization algorithm based on Adam with decoupled weight decay.

8-bit AdamW: uses a compact 8-bit representation for optimizer states to reduce memory usage. It does not mean that the LoRA parameters or logits are automatically 8-bit.

Adam states: Adam tracks first-moment and second-moment estimates of gradients to adapt the update for each parameter.

```
Gradients from backpropagation
↓
8-bit AdamW
↓
Update LoRA A and B
↓
New LoRA parameters
```

Key point: the original Qwen weights remain frozen. The effective transformation during future forward passes uses the unchanged base weight plus the newly learned LoRA update.

Why next? One update only learns a tiny amount. The same forward → loss → backward → optimizer cycle must repeat over batches and epochs.

14. Batches, Steps, Epochs and Training Completion

Batch: a small group of training examples processed together in one forward/backward pass.

Training step: one optimizer update, usually after processing one batch (or after gradient accumulation, if used).

Epoch: one complete pass through the training dataset.

```
For each epoch:
  for each batch:
    forward pass
    calculate loss
    backpropagation
    AdamW update of LoRA
  repeat
```

Dataset use: the training split contains the medical Q/A examples used to learn the adapter. Validation data can be used to monitor generalization and tune settings. The test split should remain unseen for final evaluation.

Why next? Once training is complete, we save the learned adapter and tokenizer so the trained model can be reused without retraining.

15. Saving and Loading the Fine-Tuned Model

```
model.save_pretrained("./qwen-medical-qlora")
tokenizer.save_pretrained("./qwen-medical-qlora")
```

With PEFT/LoRA, this commonly saves the LoRA adapter configuration and weights rather than a complete duplicate 7B base model.

Inference structure: load the original Qwen base model, then load the saved LoRA adapter on top.

```
Base Qwen2.5-7B
+
Saved LoRA adapter
↓
Fine-tuned model
```

Why next? Now we need to see what the fine-tuned model actually generates and measure its quality.

16. Inference / Text Generation

Inference: using a trained model to produce predictions without updating its weights.

A new medical question is tokenized and passed through the same embedding → Transformer → LM head pipeline.

```
Question
↓
Tokenizer
↓
Token IDs
↓
Qwen + LoRA
↓
Logits
↓
Decoding strategy
↓
Next token
↓
append token
↓
repeat until stopping condition
```

Decoding: converts the model's token probability distribution into an actual token sequence. Greedy decoding selects the highest-probability token; sampling methods can introduce controlled randomness.

Why next? Generated answers need objective evaluation against unseen reference answers.

17. Evaluation — ROUGE-L, BERTScore, Loss and Perplexity

ROUGE-L: measures overlap using the Longest Common Subsequence (LCS), capturing ordered word-level similarity.

Precision: how much of the generated content overlaps the reference. Recall: how much of the reference is covered. ROUGE-L combines these into an F-score.

BERTScore: compares contextual embeddings of generated and reference text, so semantically similar wording can score well even when exact words differ.

Loss: cross-entropy measures how well the model assigns probability to the correct target tokens.

Perplexity: $\exp(\text{average loss})$; lower values generally indicate better predictive fit to the evaluated text.

Example: Reference = "The patient has difficulty breathing." Generated = "The patient experiences shortness of breath." Exact overlap is limited, but BERTScore can recognize semantic similarity.

Why next? Fine-tuning changes model behavior, but it does not give the system a dedicated searchable knowledge store. RAG adds retrieval of relevant medical information at query time.

18. RAG — Retrieval-Augmented Generation

Definition: RAG combines retrieval of relevant external text with LLM generation.

```
User question
↓
Retrieve relevant medical context
↓
Question + context
↓
Fine-tuned Qwen
↓
Answer
```

Why RAG? An LLM can hallucinate. Supplying relevant retrieved context gives the model evidence to use when generating the answer.

Fine-tuning vs RAG: fine-tuning mainly teaches behavior/domain style; RAG supplies relevant information dynamically at inference time.

Why next? Before retrieval can happen, our medical knowledge must be split into searchable units.

19. RAG Step 1 — Chunking

Chunk: a smaller piece of a document used as one retrievable knowledge unit.

Large documents are divided into chunks because sending an entire document to the LLM for every question is inefficient and reduces retrieval precision.

```
Large medical document
↓
Chunk 1
Chunk 2
Chunk 3
...
Chunk N
```

Chunk size: controls how much information each unit contains. **Overlap:** optionally repeats a small number of tokens between neighboring chunks so information at a boundary is less likely to be lost.

Example: a long anemia article can become chunks about definition, symptoms, causes, diagnosis, and treatment.

Dataset use: if MedQuAD answers are used as the knowledge corpus, each answer can be treated as a small knowledge unit; for larger documents, further chunking is appropriate.

Why next? FAISS searches vectors, not raw text. Each chunk must therefore be converted into an embedding.

20. RAG Step 2 — Embeddings for Chunks

Embedding model: a model that maps text into a dense vector intended to capture semantic information.

```
Chunk text
↓
Embedding model
↓
Vector
[0.21, -0.13, 0.87, ...]
```

Semantically related chunks should have vectors that are close under the chosen similarity measure.

Important distinction: the Qwen input embedding layer converts token IDs into vectors for Qwen's Transformer. RAG embeddings are a separate representation used for *search/retrieval*. They do not have to be the same model.

Why next? Once every chunk has a vector, we can index those vectors for fast similarity search.

21. RAG Step 3 — FAISS Vector Database / Index

FAISS: Facebook AI Similarity Search, a library for efficient nearest-neighbor search over vectors.

```
Chunk 1 → vector 1
Chunk 2 → vector 2
Chunk 3 → vector 3
...
↓
FAISS
↓
similarity index
```

Why FAISS? Comparing a query against thousands or millions of vectors one-by-one can be expensive. FAISS provides optimized indexing/search methods.

Similarity: depending on the index, retrieval can use inner product, cosine-style similarity, or Euclidean distance. The configuration must match the embedding normalization/metric assumptions.

Metadata: in a practical RAG system, keep a mapping from each vector's index position back to its original chunk/document ID and text.

Why next? The index is built offline. During a real user interaction, we now need to turn the user's question into a query vector and search the index.

22. RAG Step 4 — Query Embedding and Retrieval

User asks: "What are the common causes of anemia?"

```
User question
↓
Same RAG embedding model
↓
Query vector
↓
FAISS similarity search
↓
Top-K nearest chunks
```

Top-K: the K highest-ranked retrieved chunks, such as K=3 or K=5. K is a design choice and should be evaluated.

Example results: Chunk 17 — iron deficiency; Chunk 42 — blood loss; Chunk 51 — vitamin B12 deficiency.

Why next? Retrieval only finds evidence. The LLM still needs that evidence inserted into its prompt so it can generate a coherent answer.

23. RAG Step 5 — Build the Prompt with Retrieved Context

The retrieved chunks are combined with the user's question.

```
SYSTEM:
Answer using the provided medical context.

CONTEXT:
[Chunk 17]
[Chunk 42]
[Chunk 51]

QUESTION:
What are the common causes of anemia?
```

Context window: the total amount of tokenized information the model can process for a request. Retrieval should return enough evidence without overflowing the context window.

Why next? The combined question + context is now ready for Qwen to perform generation.

24. RAG Step 6 — Generation with Qwen + LoRA

The fine-tuned Qwen receives both the user question and retrieved medical context.

```
Question + retrieved context
↓
Qwen tokenizer
↓
Qwen embeddings
↓
Transformer blocks
↓
LoRA-modified effective layers
↓
LM head
↓
Logits
↓
Vocabulary probabilities
↓
Decoding
↓
Final answer
```

Important: RAG does not retrain Qwen for every question. The model is used in inference mode; retrieval supplies context dynamically.

Why next? A usable project needs a user-facing interface that sends questions through the retrieval + generation pipeline.

25. Gradio — User Interface

Definition: Gradio is a Python framework for quickly exposing machine-learning functions through a web UI.

```
User types question
↓
Gradio event/function
↓
Query embedding
↓
FAISS retrieval
↓
Build prompt
↓
Qwen generation
↓
Return answer
↓
Gradio displays answer
```

What Gradio does not do: it is not the LLM, not the vector database, and not the RAG algorithm. It is the interface layer that connects the user to the Python inference function.

Why next? To make the project accessible remotely, the application and model need to be hosted on a compute platform.

26. Deployment — Hugging Face / Cloud

Deployment: making the trained application available on a remote server so users can access it through a URL.

For a demo, a Hugging Face Space can host the Gradio application. A ZeroGPU Space can provide temporary GPU execution under the applicable free-account quota; paid hardware provides more compute/availability.

```
Local project
■■■ app.py
■■■ requirements.txt
■■■ model / LoRA adapter reference
■■■ README / Space configuration
↓
Hugging Face Space
↓
Gradio server
↓
GPU-backed inference
↓
Public URL
```

Important for your QLoRA model: the deployment environment must have enough GPU memory for the selected Qwen model and quantization setup. The adapter can be loaded on top of the base model rather than storing a second full 7B model.

Alternative production deployment: AWS EC2 GPU, SageMaker, or another GPU service can host the same Python inference stack. Gradio is optional in AWS; FastAPI/vLLM or another API layer can be used instead.

Why deployment is last: deployment depends on a working trained model, retrieval pipeline, generation function, dependencies, and UI. It is the final packaging/serving layer.

27. Complete End-to-End MedAI Flow

TRAINING / MODEL ADAPTATION

MedQuAD Q/A
↓
Clean + deduplicate
↓
Chat template
↓
Qwen tokenizer
↓
Token IDs + masks
↓
Qwen embedding layer
↓
Positional information / RoPE
↓
Transformer blocks
↓
Self-attention (Q/K/V)
↓
4-bit frozen base weights + LoRA adapters
↓
Final hidden states
↓
LM head
↓
Logits
↓
Vocabulary softmax
↓
Cross-entropy loss
↓
Backpropagation
↓
LoRA gradients
↓
8-bit AdamW
↓
Update LoRA A/B only
↓
Repeat over batches/epochs
↓
Save adapter + tokenizer

RAG KNOWLEDGE INDEX

Medical documents / Q&A knowledge
↓
Chunking
↓
RAG embedding model
↓
Dense vectors
↓
FAISS index
↓
Store chunk ↔ vector mapping

ONLINE INFERENCE

User question
↓
RAG query embedding
↓
FAISS similarity search
↓
Top-K relevant chunks
↓
Question + retrieved context
↓
Qwen2.5-7B + LoRA
↓
Qwen tokenizer → embeddings → Transformer
↓
LM head → logits → probabilities
↓

```
Decoding
↓
Final medical answer
↓
Gradio UI
↓
User
```

28. Critical Interview Distinctions

Term	Meaning	Role in MedAI
Token	Text unit used by the LLM tokenizer	Input representation
Token ID	Integer index of a token in Qwen vocabulary	Indexes embedding matrix
Embedding	Dense vector representation	Qwen input representation / RAG search representation
Q/K/V	Query, Key, Value activations	Self-attention computation
WQ/WK/WV	Learned weight matrices	Transform hidden states into Q/K/V
Hidden state	Contextual vector at a Transformer layer	Carries contextual information
LM head	Projection to vocabulary space	Produces logits
Logit	Raw vocabulary score	Input to vocabulary softmax
Attention softmax	Attention distribution	Chooses how much tokens attend to positions
Vocabulary softmax	Next-token probability distribution	Chooses likely next token
4-bit quantization	Compact base-weight representation	Reduces memory
LoRA	Low-rank trainable update	Learns task/domain adaptation
AdamW	Adaptive optimizer with decoupled weight decay	Updates trainable parameters
8-bit AdamW	Memory-efficient optimizer-state representation	Reduces optimizer memory
RAG	Retrieve + generate	Adds external context at inference
Chunk	Retrievable text unit	Searchable knowledge unit
FAISS	Vector similarity search library	Finds relevant chunks
Gradio	ML web UI framework	User-facing interface

29. 60-Second Interview Explanation

"I built a medical Q&A chatbot using Qwen2.5-7B-Instruct. I cleaned and formatted MedQuAD question-answer pairs using Qwen's chat template and tokenized them with the Qwen tokenizer. Token IDs are mapped through Qwen's learned embedding layer and passed through the Transformer blocks. For efficient fine-tuning, I used QLoRA: the pretrained base model is loaded in 4-bit precision and kept frozen, while small LoRA adapters are inserted into selected linear layers. During training, the model performs a forward pass, produces vocabulary logits, converts them to probabilities, calculates cross-entropy loss, backpropagates gradients to the LoRA parameters, and uses 8-bit AdamW to update only those trainable adapter weights. After training, I save the adapter and evaluate generated answers using metrics such as ROUGE-L and BERTScore. I then add RAG: medical text is chunked, embedded with a retrieval embedding model, and indexed in FAISS. At inference, the user's question is embedded, relevant chunks are retrieved, and those chunks are supplied as context to the fine-tuned Qwen model before generation. Finally, I expose the pipeline through a Gradio interface and can deploy it on Hugging Face or another GPU cloud."

30. Final Mental Model

Remember the separation:

```
Qwen input embeddings
→ represent tokens for the Transformer

RAG embeddings
→ represent documents/questions for retrieval

4-bit quantization
```

→ reduces memory used by frozen base weights

LoRA

→ small trainable update to selected model layers

8-bit AdamW

→ reduces optimizer-state memory and updates LoRA parameters

LM head

→ converts final hidden states into vocabulary logits

Vocabulary softmax

→ converts logits into next-token probabilities

RAG

→ retrieves external context before generation

FAISS

→ searches the vector index

Gradio

→ provides the user interface

Note: Exact Qwen layer counts, target modules, quantization configuration, embedding model, chunk size, overlap, FAISS index type, and deployment hardware should match the actual code used in your final repository. Do not claim a component on a resume unless it is implemented and tested in the project.